

0x8: Browser-native Post-Quantum Cryptographic Suite

Complete Protocol Specification

Abstract

0x8: Browser-native Post-Quantum Cryptographic suite (Video Call, Chat, File Transfer) implementing NIST-standardized algorithms at the highest security level:

- ML-KEM-1024 (FIPS 203) • Kyber-1024,
- ML-DSA-87 (FIPS 204) • Dilithium-87,
- SLH-DSA-256 (FIPS 205) • Sphincs+,
- FN-DSA-1024 (FIPS 206) • Falcon → Algorithm is in the process of final standardization by NIST.

The system provides hybrid encrypt-then-sign file/message encryption with a structured binary container format (PQENC), a commit-reveal key exchange protocol with Short Authentication String (SAS) verification for WebRTC peer-to-peer connections, forward-secret ratcheted session encryption for real-time chat and file transfer, and post-quantum media encryption for video calls using WebRTC Insertable Streams. This report provides a complete specification of the protocols, algorithms, data formats, and security properties, including a formal analysis of attack surfaces and computational difficulty for adversaries.

1 Introduction

1.1 Motivation

The transition to post-quantum cryptography (PQC) demands not only standardized algorithms but also practical, deployable systems that integrate PQC primitives into existing communication infrastructures. The “Store-Now-Decrypt-Later” (SNDL) threat makes classical key agreement (ECDH) insufficient: an adversary recording ciphertexts today can decrypt them once a cryptographically relevant quantum computer emerges. Browser-based deployment offers zero-install accessibility but introduces unique challenges in trust establishment, key management, and real-time encryption.

1.2 Contributions

0x8 provides:

- A hybrid encrypt-then-sign protocol (PQENC) combining ML-KEM-1024 key encapsulation with AES-256-GCM authenticated encryption and NIST Level 5 digital signatures, with the signature itself bound into the Additional Authenticated Data (AAD).
- A commit-reveal handshake protocol with Proof-of-Possession (PoP) signatures and SAS-based out-of-band verification, preventing man-in-the-middle attacks even against a malicious signaling server.

- A forward-secret ratcheted session layer using HKDF-SHA-256 per-message key derivation with separate send/receive ratchet chains, rollback detection, and automatic PQC re-keying.
- Post-quantum media encryption via WebRTC Insertable Streams and a Web Worker, applying AES-256-GCM to RTP frames with time-based autonomous ratcheting (3-second intervals).
- A structured binary container format with magic bytes, versioning, algorithm identifiers, and length-prefixed fields for unambiguous parsing and integrity verification.

1.3 Threat Model

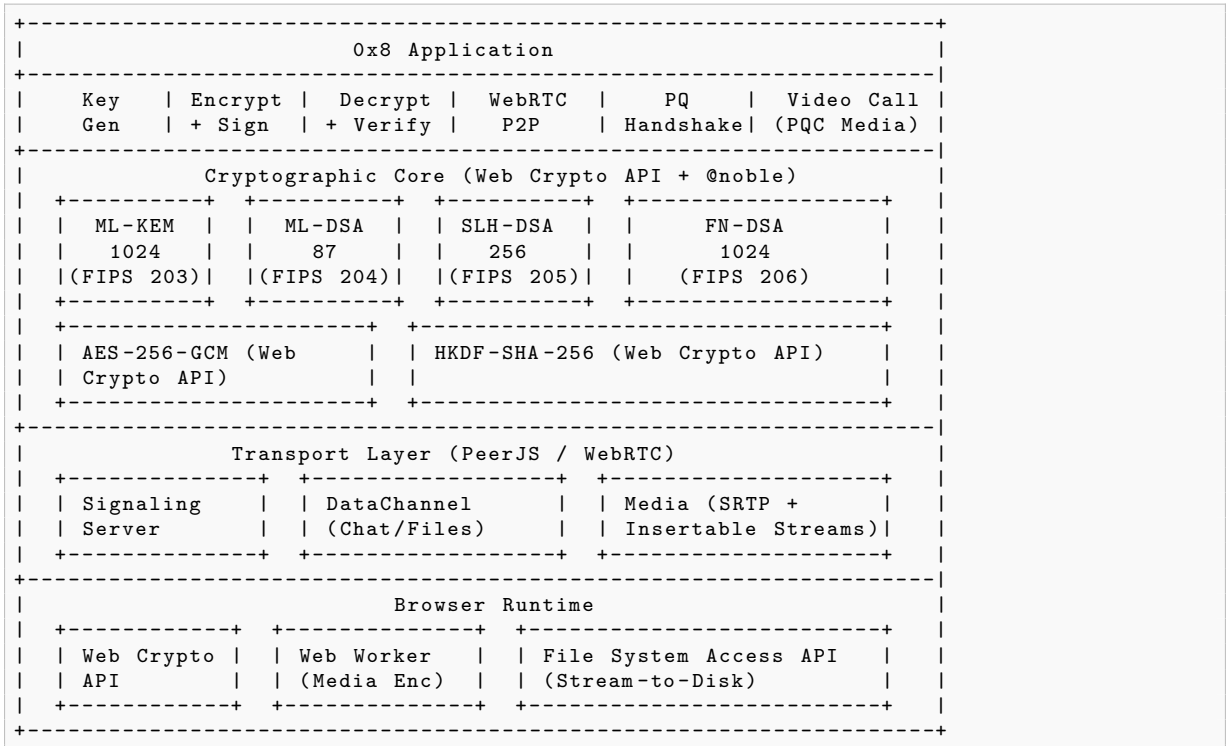
We consider adversaries with the following capabilities:

- **Network adversary:** Full control over the signaling server and network paths (MITM).
- **Quantum adversary:** Possession of a cryptographically relevant quantum computer capable of breaking ECDH, RSA, and ECDSA.
- **Classical adversary:** Bounded polynomial-time classical computation.
- **Insider threat:** One party attempting to forge messages from the other.

We assume the out-of-band channel for SAS verification is authentic but not confidential.

2 System Architecture

2.1 Component Overview



2.2 Library Loading Strategy

The application dynamically imports PQC libraries using ES module imports with an importmap resolving to local @noble/post-quantum builds:

```
importmap:
  @noble/hashes/          ./@noble/hashes/
  @noble/curves/          ./@noble/curves/
  @noble/ciphers/         ./@noble/ciphers/
  @noble/post-quantum/    ./@noble/post-quantum/
```

All `<script>` and `<link rel="modulepreload">` elements include Subresource Integrity (SRI) hashes (SHA-512), ensuring that any tampering with the library files is detected by the browser before execution.

The loading strategy performs a test `keygen()` for each algorithm to verify that the library exports are functional before marking the application as ready.

3 Key Generation

3.1 Supported Algorithms and Parameters

Algorithm	NIST Standard	Public Key Size	Private Key Size
ML-KEM-1024	FIPS 203	1,568 bytes	3,168 bytes
ML-DSA-87	FIPS 204	2,496 bytes	4,896 bytes
SLH-DSA-SHA2-256s	FIPS 205	64 bytes	128 bytes
SLH-DSA-SHA2-256f	FIPS 205	64 bytes	128 bytes
SLH-DSA-SHAKE-256s	FIPS 205	64 bytes	128 bytes
SLH-DSA-SHAKE-256f	FIPS 205	64 bytes	128 bytes
FN-DSA-1024	FIPS 206	1,793 bytes	2,305 bytes

For SLH-DSA variants: **s** (small/compact signatures, slower signing) vs. **f** (fast signing, larger signatures); SHA2 (SHA-256/512 core) vs. SHAKE (SHAKE256/SHA-3 core, providing hash diversity).

3.2 Key Regeneration and Handshake Reset

If a key pair is regenerated while a P2P connection is active and a PQ handshake has been performed (or is in progress), the system:

- Aborts any in-progress file transfers.
- Sends a `pq-handshake-reset` message to the peer.
- Resets the local handshake state (commitments, peer keys, SAS, trust status).
- Disables the PQC session encryption layer.

This prevents key confusion attacks where stale keys could be used after regeneration.

4 PQENC: Encrypt-then-Sign Protocol

4.1 Binary Container Format

The PQENC format encodes all cryptographic artifacts into a single binary packet with a fixed header structure and length-prefixed variable fields:

Offset	Size	Field	Description
0	5	Magic	“PQENC” (ASCII)
5	1	Version	0x08
6	1	SigAlgoId	0x01 = ML-DSA-87, 0x02 = SLH-DSA-SHA2-256s, 0x03 = SLH-DSA-SHA2-256f, 0x04 = SLH-DSA-SHAKE-256s, 0x05 = SLH-DSA-SHAKE-256f, 0x06 = FN-DSA-1024
7	8	Timestamp	Unix epoch (BigUInt64, big-endian)
15	16	Nonce	Random (cryptographically secure)
31	2	KemCtLen	Length of KEM ciphertext (uint16, BE)
33	var	KemCt	ML-KEM-1024 ciphertext (1,568 bytes)
33+ L_k	1	IvLen	Length of IV (always 12)
34+ L_k	12	IV	AES-256-GCM initialization vector
46+ L_k	4	EncDataLen	Length of encrypted data (uint32, BE)
50+ L_k	var	EncData	AES-256-GCM ciphertext + 16-byte auth tag
50+ L_k+L_e	2	SigLen	Length of signature (uint16, BE)
52+ L_k+L_e	var	Signature	PQC digital signature

Total minimum size (empty message, ML-DSA-87): $52 + 1,568 + 16 + 4,627 = 6,263$ bytes.

Design rationale: Length prefixes allow unambiguous parsing even without knowing the signature algorithm in advance—the **SigAlgoId** field identifies the algorithm, and the **SigLen** field locates the signature boundary. The magic bytes and version enable future format evolution.

4.2 Additional Authenticated Data (AAD) Construction

The AAD binds all metadata and the signature itself into the AES-GCM authentication tag, ensuring that any modification to any field is detected:

AAD = Version (1 byte)
 ||SigAlgoId (1 byte)
 ||Timestamp (8 bytes, big-endian)
 ||Nonce (16 bytes)
 ||KemCtLen (2 bytes, big-endian)
 ||KemCt (KemCtLen bytes)
 ||IvLen (1 byte)
 ||EncDataLen (4 bytes, big-endian)
 ||SigLen (2 bytes, big-endian)
 ||Signature (SigLen bytes)

Critical design property: The digital signature is included in the AAD. This creates a bidirectional integrity binding:

- AES-GCM authentication ensures the signature was not modified in transit.
- The DSA signature covers (plaintext||KEM ciphertext), ensuring message authenticity.
- Modifying either the signature or the ciphertext causes AES-GCM tag verification to fail, preventing selective tampering.

4.3 Encryption Algorithm

Algorithm 1: PQENC-Encrypt

Input: Plaintext P , Recipient ML-KEM pk_R , Sender DSA sk_S , SigAlgo ID

Output: PQENC binary packet

```

1 ( $ct_{KEM}, ss$ )  $\leftarrow$  ML-KEM-1024.Encapsulate( $pk_R$ )           //  $ss$  = shared secret (32
   bytes),  $ct_{KEM}$  = KEM ciphertext (1,568 bytes);
2  $ts \leftarrow$  CurrentTime()/1000 // Unix timestamp (seconds)
3 ;
4  $nonce \leftarrow$  RandomBytes(16) // Cryptographically secure
5 ;
6  $s \leftarrow$  DSA.Sign( $sk_S, P || ct_{KEM}$ ) // Sign plaintext || KEM ciphertext
7 ;
8  $AAD \leftarrow$  ConstructAAD(version =
   0x08, SigAlgoId,  $ts, nonce, len(ct_{KEM}), ct_{KEM}, IV\_len = 12, len(P) + 16, len(s), s$ );
9  $AES\_key \leftarrow$  HKDF-SHA-256(IKM =  $ss$ , salt =  $ct_{KEM}$ , info =
   "PQENC-AES256-GCM",  $L = 256$  bits);
10  $IV \leftarrow$  RandomBytes(12);
11 ( $C, tag$ )  $\leftarrow$  AES-256-GCM-Encrypt( $AES\_key, IV, P, AAD$ );
12  $encData \leftarrow C || tag$  //  $len = len(P) + 16$ 
13 ;
14 return PackPQENC(version, SigAlgoId,  $ts, nonce, ct_{KEM}, IV, encData, s$ )

```

4.4 Decryption Algorithm

Algorithm 2: PQENC-Decrypt

Input: PQENC binary packet, Recipient ML-KEM sk_R , Sender DSA pk_S

Output: Plaintext P , signature validity bit

```

1 Parse packet  $\rightarrow$  extract: version, SigAlgoId,  $ts, nonce, ct_{KEM}, IV, encData, s$ ;
   // Reject if magic  $\neq$  "PQENC" or version  $\neq$  0x08
2 Validate:
3   (a)  $len(ct_{KEM}) = 1,568$ ;
4   (b)  $len(IV) = 12$ ;
5   (c)  $|CurrentTime()/1000 - ts| \leq 180$  //  $\pm 3$  minute window
6 ;
7  $ss \leftarrow$  ML-KEM-1024.Decapsulate( $ct_{KEM}, sk_R$ );
8  $AES\_key \leftarrow$  HKDF-SHA-256(IKM =  $ss$ , salt =  $ct_{KEM}$ , info =
   "PQENC-AES256-GCM",  $L = 256$  bits);
9  $AAD \leftarrow$ 
   ReconstructAAD(version, SigAlgoId,  $ts, nonce, len(ct_{KEM}), ct_{KEM}, 12, len(encData), len(s), s$ );
10  $P \leftarrow$  AES-256-GCM-Decrypt( $AES\_key, IV, encData, AAD$ ) // Reject if tag
   verification fails
11 ;
12  $valid \leftarrow$  DSA.Verify( $s, P || ct_{KEM}, pk_S$ );
13 return ( $P, valid$ )

```

4.5 Key Derivation Details

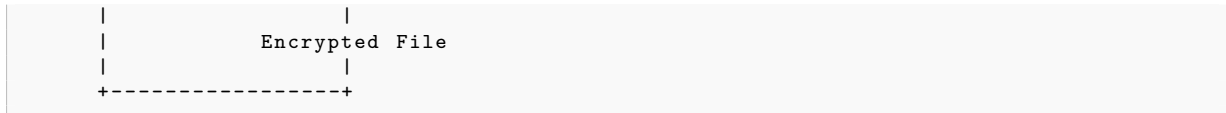
The AES-256-GCM key is derived using HKDF (HMAC-based Extract-and-Expand Key Derivation Function) with SHA-256:

HKDF-SHA-256:

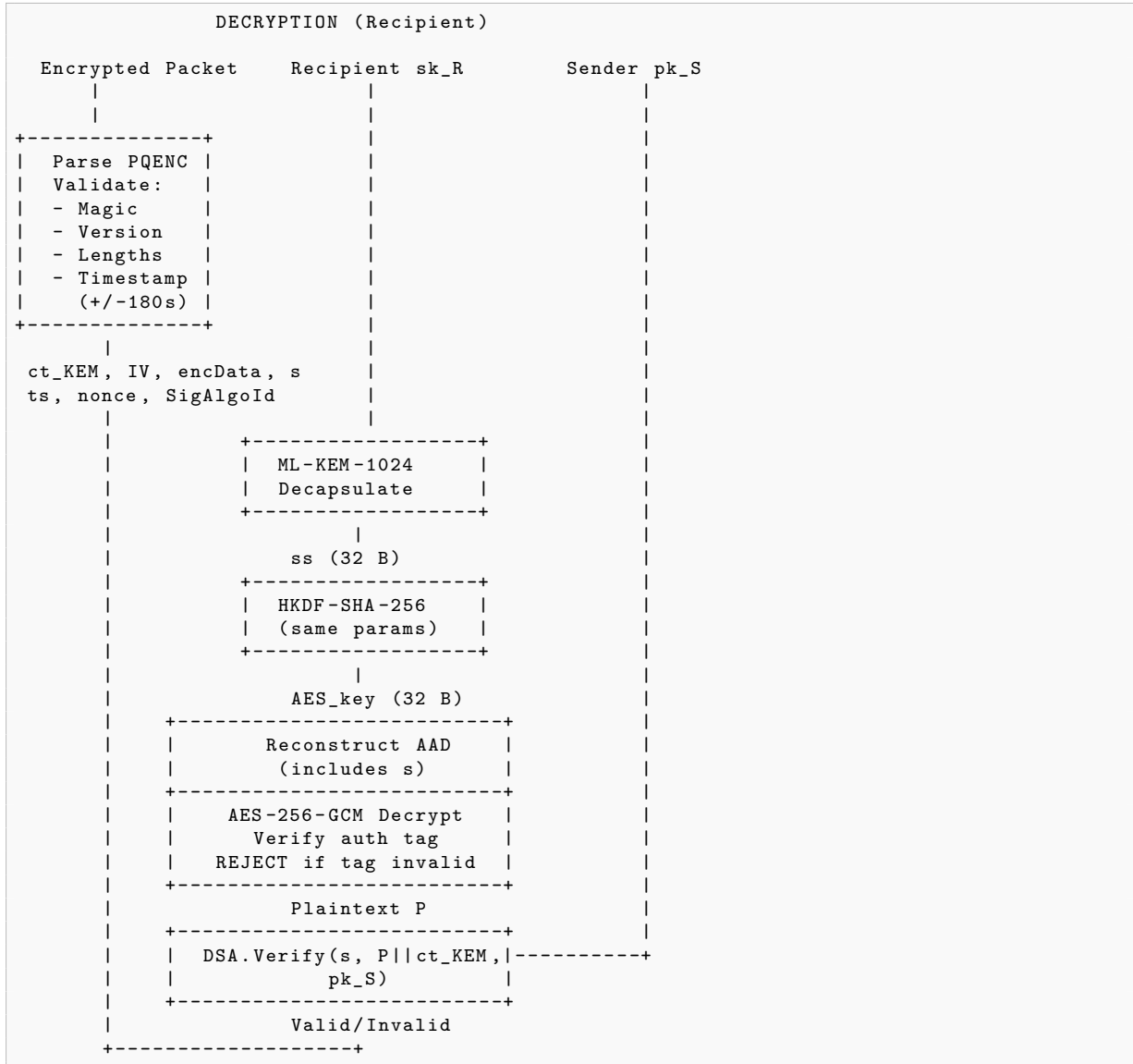
- **Extract:** $PRK = \text{HMAC-SHA-256}(\text{salt} = \text{ct}_{\text{KEM}}, \text{IKM} = ss)$
- **Expand:** $OKM = \text{HMAC-SHA-256}(PRK, \text{info} || 0x01)$
- $\text{AES_key} = \text{OKM}[0..32]$
- **IKM (Input Keying Material):** The ML-KEM shared secret (32 bytes).
- **Salt:** The KEM ciphertext (1,568 bytes). This ensures key uniqueness per encapsulation, even if the same shared secret were somehow produced twice (computationally infeasible).
- **Info:** The ASCII string “PQENC-AES256-GCM”. This domain-separates the derived key from any other use of the same shared secret in other protocols.

4.6 Encryption Flow Diagram





4.7 Decryption Flow Diagram



5 PQ Handshake Protocol

5.1 Motivation

WebRTC uses DTLS with classical ECDH for key agreement. A quantum adversary performing SNDL can record the DTLS handshake today and derive session keys later. More critically, a malicious signaling server can perform a MITM attack by substituting DTLS keys during connection establishment. The PQ Handshake protocol provides post-quantum authentication of the peer's FIPS identity keys, independent of the WebRTC DTLS layer.

5.2 Protocol Specification

The protocol involves two parties, Alice (A) and Bob (B), connected via WebRTC DataChannel through an untrusted signaling server.

Phase 1: COMMIT

$A \rightarrow B$: type = “pq-commitment”
commitment_A = SHA-256(salt_A||pk_{KEM,A}||pk_{DSA,A}||algoId_A)
 $B \rightarrow A$: type = “pq-commitment”
commitment_B = SHA-256(salt_B||pk_{KEM,B}||pk_{DSA,B}||algoId_B)

Phase 2: REVEAL (after both commitments received)

$A \rightarrow B$: type = “pq-keys-reveal”
kemPub = pk_{KEM,A} (Base64), dsaPub = pk_{DSA,A} (Base64)
sigAlgo = algoId_A, salt = salt_A (Base64)
popSignature = DSA.Sign(sk_{DSA,A}, pk_{KEM,A}||pk_{DSA,A}||salt_A||algoId_A)
 $B \rightarrow A$: type = “pq-keys-reveal”
kemPub = pk_{KEM,B} (Base64), dsaPub = pk_{DSA,B} (Base64)
sigAlgo = algoId_B, salt = salt_B (Base64)
popSignature = DSA.Sign(sk_{DSA,B}, pk_{KEM,B}||pk_{DSA,B}||salt_B||algoId_B)

Phase 3: VERIFY

Each party:

1. Recompute SHA-256(salt_{peer}||pk_{KEM,peer}||pk_{DSA,peer}||algoId_{peer})
2. Compare with received commitment (constant-time)
3. Verify PoP signature: DSA.Verify(popSignature, pk_{KEM}||pk_{DSA}||salt||algoId, pk_{DSA,peer})

Phase 4: SAS GENERATION

```
SAS_input = sorted([
    JSON({algo: algoId_A, dsa: b64(pk_DSA_A), id: id_A, kem: b64(pk_KEM_A)}),
    JSON({algo: algoId_B, dsa: b64(pk_DSA_B), id: id_B, kem: b64(pk_KEM_B)})
]).join('')

SAS = SHA-256(SAS_input)[0..16 hex chars].toUpperCase()
```

Phase 5: OUT-OF-BAND VERIFICATION

Alice and Bob compare SAS via a trusted channel (e.g., phone call).

- If SAS matches → trustPeer()
- If SAS mismatches → distrustPeer()

Phase 6: TRUST ESTABLISHMENT

On trust:

- Lock peer keys as verified
- Auto-populate Encrypt/Decrypt tabs with peer keys
- Enable PQC session encryption toggle

5.3 Commitment Verification (Constant-Time)

The commitment comparison uses a constant-time XOR reduction to prevent timing side-channel attacks:

```
// Constant-time comparison
let mismatch = hashBytes.length !== commitmentBytes.length ? 1 : 0;
for (let i = 0; i < hashBytes.length; i++)
    mismatch |= hashBytes[i] ^ commitmentBytes[i];
if (mismatch) -> REJECT
```

5.4 SAS Computation Detail

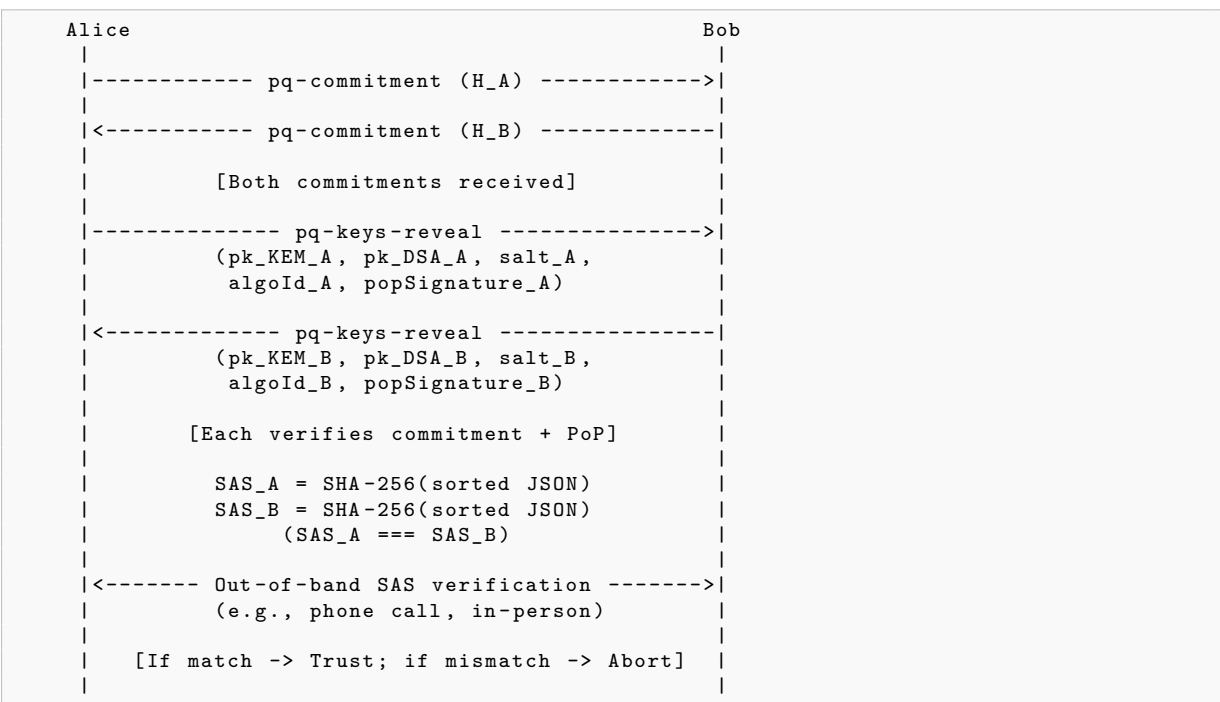
The SAS is computed by:

1. Creating a canonical JSON representation of each peer's identity (keys, algorithm, ID).
2. Sorting the two JSON strings lexicographically.
3. Concatenating them.
4. Computing SHA-256 of the concatenation.
5. Taking the first 16 hexadecimal characters (64 bits) of the digest.

The deterministic sorting ensures both parties compute the same SAS without coordination.

Security of 64-bit SAS: The probability that a MITM attacker can forge a matching SAS is 2^{-64} , which is computationally negligible. For a single verification attempt, this provides approximately 64 bits of security against a MITM attack on the SAS itself.

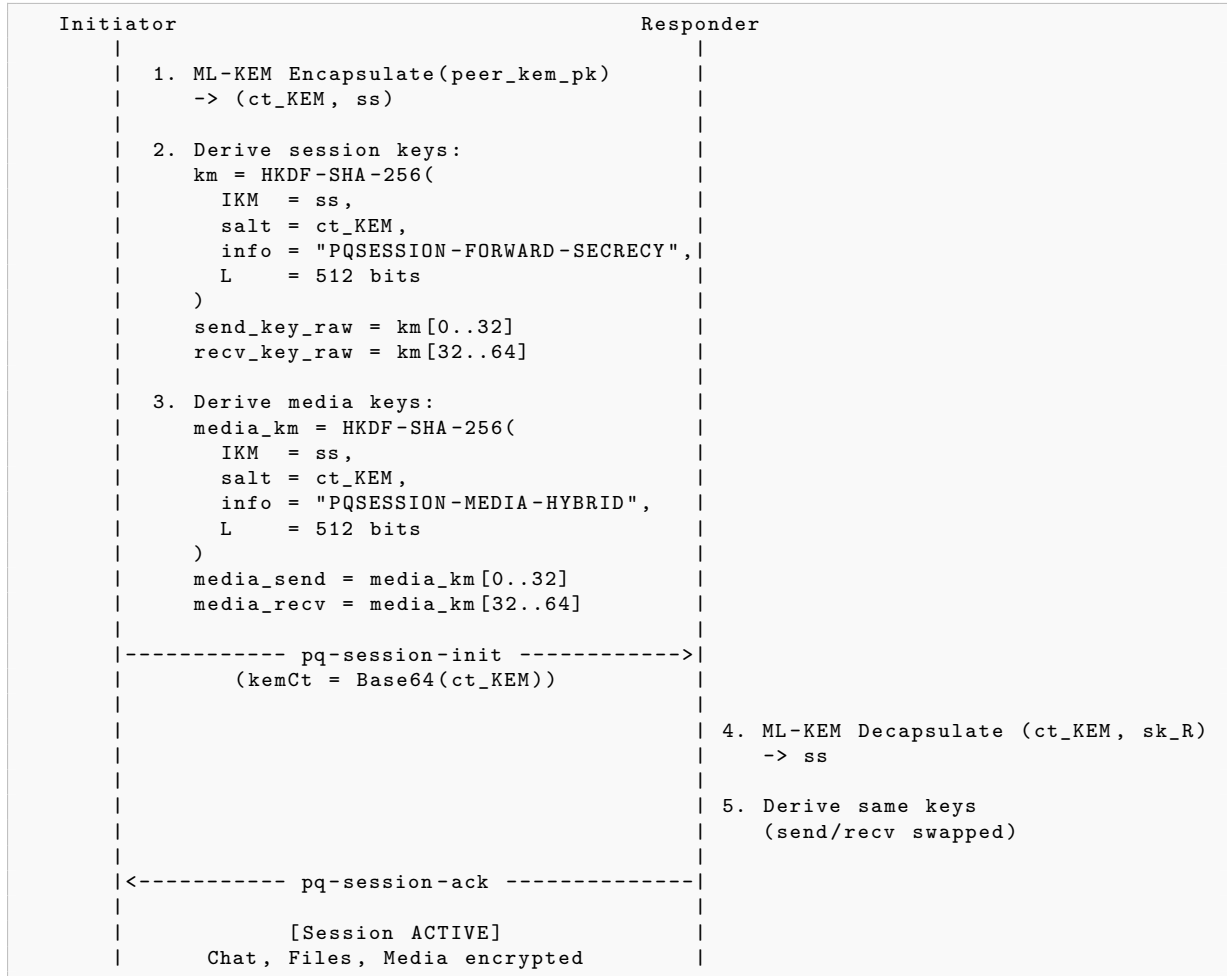
5.5 Handshake Flow Diagram



6 PQC Session Encryption

6.1 Session Establishment

After PQ Handshake trust is established, either party may initiate PQC session encryption:



Key assignment convention: The initiator's `send_key_raw` becomes the responder's `recv_key_raw`, and vice versa. This is achieved by the responder swapping the first and second 32-byte halves.

6.2 Per-Message Ratcheting (Forward Secrecy)

Each sent or received message advances the ratchet state independently:

Ratchet Key Derivation:

```
ratchet_key(current_key, counter) :
    salt = uint32_big_endpoint(counter)
    return HKDF-SHA-256(
        IKM = current_key,
        salt = salt,
        info = "PQSESSION-RATCHET",
        L = 256 bits
    )
```

Send ratchet: Before each send, `pqSendCounter ++`, then `pqSendKeyRaw = ratchet_key(pqSendKeyRaw, pqSendCounter)`. A new AES-GCM key object is created from `pqSendKeyRaw`.

Receive ratchet: Upon receiving a message with counter `cnt`, if `cnt > pqRecvCounter`, the receiver fast-forwards its ratchet state by repeatedly applying `ratchet_key()` until `pqRecvCounter === cnt`. If `cnt < pqRecvCounter` (and the difference is within the valid range), the message is rejected as a rollback attempt.

Rollback detection:

```
if (data.cnt < pqRecvCounter &&
    (pqRecvCounter - data.cnt) < 231) {
    // Rollback detected -> REJECT
}
```

6.3 Encrypted Chat Message Format

```
{
  type: "pq-chat",
  iv:   Base64(IV),           // 12 bytes, random
  data: Base64(encData),      // AES-256-GCM ciphertext + tag
  cnt:  uint32                // Ratchet counter
}
```

AAD for chat messages: The ratchet counter is used as AAD:

`AAD = uint32_big_endpoint(cnt)`

This binds each message to its ratchet position, preventing counter manipulation.

6.4 Encrypted File Chunk Format

```
{
  type: "pq-chunk",
  iv:   Base64(IV),           // 12 bytes, random
  data: Base64(encData),      // AES-256-GCM ciphertext + tag
  cnt:  uint32                // Ratchet counter
}
```

File chunks use the same ratchet mechanism as chat messages, advancing the send key after each chunk.

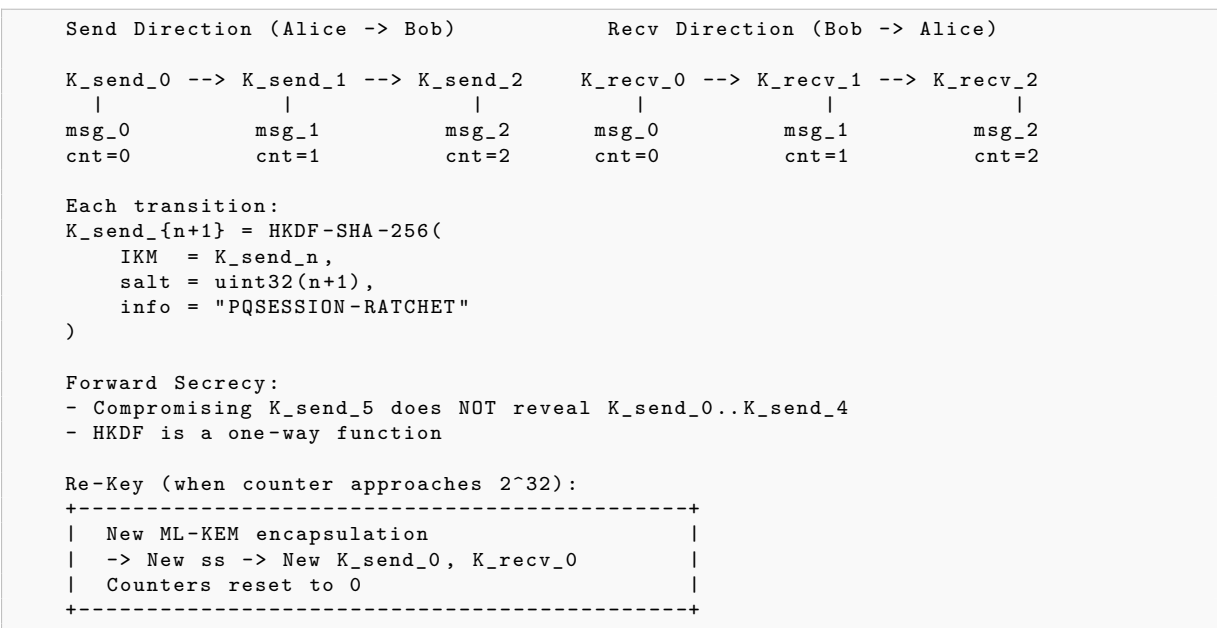
6.5 PQC Re-Keying

When the ratchet counter approaches its limit ($2^{32} - 1$), the system performs a full PQC re-key:

- The requesting party performs a fresh ML-KEM encapsulation with the peer's verified KEM public key.
- New session keys and media keys are derived from the new shared secret.
- Counters are reset to zero.
- A `pq-session-init` message is sent with the new KEM ciphertext.
- The peer decapsulates and derives the same new keys.

This provides post-compromise security: even if a session key is somehow compromised, the re-keying establishes fresh forward secrecy from that point forward.

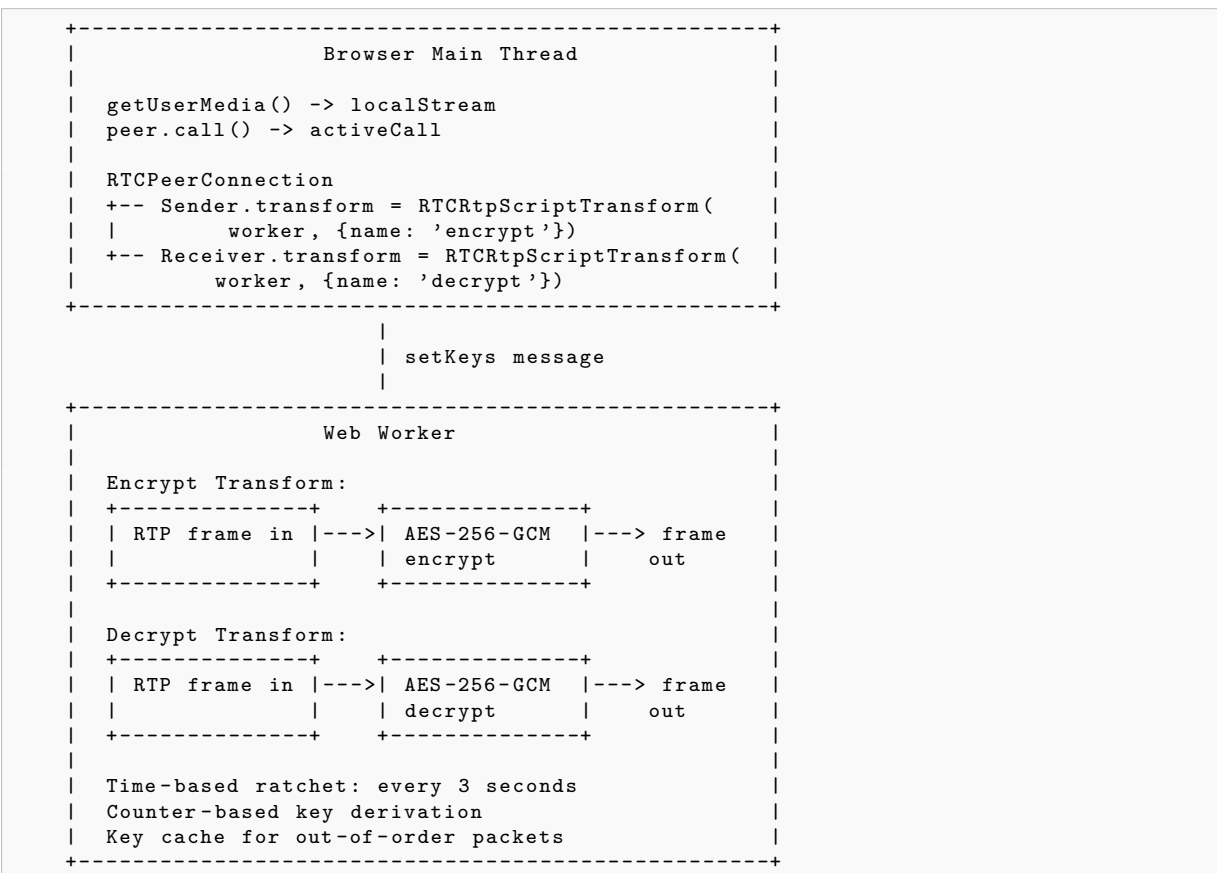
6.6 Session Ratchet Diagram



7 PQC Media Encryption (Video Calls)

7.1 Architecture

Video call encryption uses WebRTC Insertable Streams (`RTCRtpScriptTransform`) to apply PQC encryption directly on RTP frames, independent of the SRTP/DTLS layer:



7.2 Encrypted Frame Format

Ratchet ID	IV	AES-256-GCM Ciphertext
(4 bytes)	(12 bytes)	+ Auth Tag (16 bytes)
uint32 BE	random	

- **Ratchet ID (4 bytes, big-endian):** The ratchet counter value used to derive the key for this frame. Sent in cleartext so the receiver can look up the correct decryption key.
- **IV (12 bytes):** Random initialization vector per frame.
- **Ciphertext:** The encrypted RTP frame data with GCM authentication tag.

7.3 Time-Based Autonomous Ratcheting

The media worker autonomously advances the encryption key every 3 seconds without coordinating with the main thread or the peer:

```
RATCHET_INTERVAL = 3000; // milliseconds

async function getEncKey() {
  const now = Date.now();
  if (now - lastEncTime >= RATCHET_INTERVAL || !currentEncKey) {
    encCounter++;
    encSeed = ratchetKey(encSeed, encCounter);
    currentEncKey = await crypto.subtle.importKey(
      'raw', encSeed, 'AES-GCM', false, ['encrypt']
    );
    lastEncTime = now;
  }
  return { key: currentEncKey, id: encCounter };
}
```

Rationale: Real-time media requires sub-millisecond latency. Waiting for key acknowledgment from the peer would introduce unacceptable delay. Instead, the receiver uses the Ratchet ID to independently derive the same key.

7.4 Decryption Key Cache

Since RTP packets may arrive out of order, the receiver maintains a key cache:

```
// Cache structure
decKeyCache[counter] = CryptoKey;

// Cache eviction: keep most recent 64 keys
if (Object.keys(decKeyCache).length > 128) {
  const sorted = Object.keys(decKeyCache).map(Number).sort((a,b)=>a-b);
  for (let i = 0; i < sorted.length - 64; i++)
    delete decKeyCache[sorted[i]];
}
```

Fast-forward mechanism: If a gap is detected between the expected counter and the received counter, the receiver fast-forwards its ratchet state up to the received counter (up to 256 steps). If the gap exceeds 256, the frame is dropped.

Anti-rollback: Frames with counter values that are clearly older than the current state are rejected.

7.5 Media Key Derivation

Media keys are derived separately from chat/file keys using a different HKDF info string, providing domain separation:

```
media_km = HKDF-SHA-256(IKM = ss, salt = ct_KEM, info = "PQSESSION-MEDIA-HYBRID",
    L = 512 bits)
media_send_seed = media_km[0..32] // Initiator: send; Responder: recv
media_recv_seed = media_km[32..64] // Initiator: recv; Responder: send
```

Media ratchet key derivation:

```
media_ratchet_key(baseSeed, counter) :
    salt = uint32_big_endian(counter)
    return HKDF-SHA-256(
        IKM = baseSeed,
        salt = salt,
        info = "PQSESSION-MEDIA-RATCHET",
        L = 256 bits
    )
```

8 File Transfer Protocol

8.1 Transfer Modes

The application supports two file transfer modes:

- **Stream-to-Disk (Chrome/Edge):** Uses the File System Access API (`showSaveFilePicker`) to write chunks directly to the user's hard drive in real-time, minimizing memory usage.
- **Stream-to-RAM (Firefox/Safari):** Buffers all chunks in RAM and offers a download link upon completion. A 2 GB RAM limit is enforced.

8.2 File Transfer Protocol Messages

Unencrypted mode:

```
Sender -> Receiver: { type: "file-start", name: string, size: number }
Sender -> Receiver: ArrayBuffer (16 KB chunks)
Sender -> Receiver: { type: "file-end" }
```

PQC-encrypted mode:

```
Sender -> Receiver: { type: "file-start", name: string, size: number }
Sender -> Receiver: { type: "pq-chunk", iv: Base64, data: Base64, cnt: uint32 }
Sender -> Receiver: { type: "file-end" }
```

Cancellation:

```
Either party: { type: "file-cancel" }
```

8.3 Flow Control

The sender monitors the DataChannel's `bufferedAmount` and pauses when it exceeds 1 MB:

```
if (bufAmt > 1_048_576) {
  setTimeout(sendNextChunk, 50);
  return;
}
```

This prevents memory exhaustion on the receiver side and ensures reliable delivery.

8.4 File Name Sanitization

Received file names are sanitized to prevent path traversal attacks:

```
wrRecvName = data.name
  .replace(/[\\\/\]/g, '_')      // Remove path separators
  .replace(/\.\./g, '')         // Remove parent directory references
  .replace(/[<>:"|?*\\0]/g, ''); // Remove invalid characters
```

8.5 Size Limits

Mode	Maximum File Size
Stream-to-Disk	2 TB (2,199,023,255,552 bytes)
Stream-to-RAM	2 GB (2,147,483,648 bytes)

Files exceeding the 2 GB limit that are not saving to disk are automatically rejected.

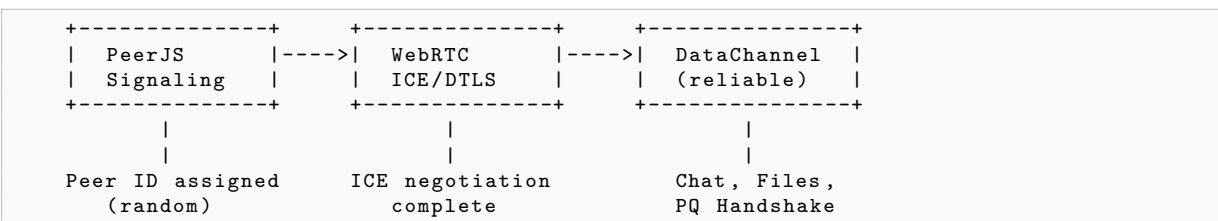
9 WebRTC Connection Management

9.1 ICE Configuration

The application provides three ICE strategies:

Strategy	Type	Servers
STUN (Google)	STUN only	stun:stun.l.google.com (multiple ports)
TURN (Open Relay)	STUN + TURN	turn:openrelay.metered.ca (UDP/TCP)
TURN (Metered)	STUN + TURN	turn:a.relay.metered.ca (UDP/TCP)

9.2 Connection Lifecycle



9.3 Connection Collision Resolution

When both peers attempt to connect simultaneously, the application uses a deterministic rule based on Peer ID comparison:

```
if (activeConn && activeConn.open) {
    if (activeConn.peer === conn.peer) {
        if (conn.peer < wrPeerId) {
            conn.close(); // Keep existing
            return;
        }
        activeConn.close(); // Replace with new
    }
}
```

The peer with the lexicographically smaller ID closes the incoming duplicate connection.

10 Security Analysis

10.1 Cryptographic Security

10.1.1 ML-KEM-1024 (FIPS 203)

ML-KEM-1024 provides NIST Security Level 5 ($> 2^{248}$ classical security, $> 2^{192}$ quantum security). The best known classical attack on ML-KEM-1024 requires approximately 2^{276} operations (primal lattice attack), and the best known quantum attack requires approximately 2^{230} operations. These costs are far beyond any conceivable computing capability.

SNDL resistance: ML-KEM-1024 is based on the Module-LWE problem, which is believed to be hard for quantum computers. An adversary recording KEM ciphertexts today cannot derive the shared secret even with a future quantum computer.

10.1.2 Digital Signature Algorithms

All supported signature algorithms provide NIST Level 5 security:

Algorithm	Assumption	Quantum Security
ML-DSA-87	Module-SIS/LWE	$> 2^{192}$
SLH-DSA-256	Collision resistance of hash functions	$> 2^{192}$
FN-DSA-1024	NTRU lattice problems	$> 2^{192}$

SLH-DSA provides conservative hash-based security with minimal assumptions, while FN-DSA-1024 offers the most compact signatures at Level 5.

10.1.3 AES-256-GCM

AES-256-GCM provides 256-bit key security and 128-bit authentication tag security. The authentication tag ensures integrity with a forgery probability of at most 2^{-128} per attempt.

Nonce management: Each encryption operation uses a cryptographically random 12-byte IV. With random IVs, the birthday bound gives a collision probability of approximately 2^{-38} after 2^{30} encryptions under the same key—well within safe limits given the per-session key derivation.

10.1.4 HKDF-SHA-256

HKDF is provably secure under the HMAC extraction and expansion framework. The use of distinct info strings provides cryptographic domain separation between:

- File encryption keys ("PQENC-AES256-GCM")
- Session chat/file keys ("PQSESSION-FORWARD-SECREC")
- Session media keys ("PQSESSION-MEDIA-HYBRID")
- Ratchet keys ("PQSESSION-RATCHET", "PQSESSION-MEDIA-RATCHET")

10.2 Protocol Security

10.2.1 PQENC Encryption Security Properties

Property	Mechanism
Confidentiality	AES-256-GCM encryption with ML-KEM-derived key
Authenticity	AES-GCM authentication tag + DSA signature
Integrity	AAD binds all metadata + signature into GCM tag
Non-repudiation	DSA signature on (plaintext KEM ciphertext)
Replay resistance	Timestamp validation (± 180 seconds)
Key uniqueness	HKDF with KEM ciphertext as salt
Algorithm agility	SigAlgId field supports 6 signature algorithms

Encrypt-then-Sign vs. Sign-then-Encrypt: The PQENC protocol uses Sign-then-Encrypt (sign the plaintext, then encrypt it), but with a critical enhancement: the signature is also included in the AAD of the encryption. This means:

- The signature is authenticated by AES-GCM (any modification to the signature in transit is detected).
- The signature covers the plaintext and KEM ciphertext (proving authorship and integrity of the plaintext).
- The KEM ciphertext in the signed data provides binding between the encryption key and the signer's claim.

Why include the signature in AAD? Without this binding, an attacker who compromises the recipient's KEM private key could:

1. Decrypt the ciphertext.
2. Replace the original signature with their own (re-signing the plaintext).
3. The recipient would not detect this substitution.

By including the signature in the AAD, any modification to the signature causes AES-GCM tag verification to fail, creating an indissoluble link between the signature's integrity and the encrypted payload.

10.2.2 Handshake Security Properties

Commit-reveal prevents adaptive MITM: In a standard key exchange, a malicious signaling server could wait for Alice to send her public key, then send a different key to Bob (and vice versa). The commit-reveal protocol forces the adversary to commit to a key before seeing the other party's key, making adaptive key selection impossible.

PoP prevents key injection: Even if an adversary obtains a valid DSA public key, they cannot produce a valid PoP signature without the corresponding private key. This prevents an attacker from claiming ownership of a key they don't control.

Property	Mechanism
MITM resistance	Commit-reveal prevents adaptive key substitution
Key ownership proof	Proof-of-Possession (PoP) signature
Out-of-band verification	SAS comparison via trusted channel
Signaling server independence	PQ handshake operates over DataChannel after WebRTC establishment

SAS provides 64-bit verification security: The 16-hex-character SAS provides 2^{64} possible values. The probability of a MITM attacker producing a matching SAS is 2^{-64} , which is negligible.

10.2.3 Session Encryption Security Properties

Property	Mechanism
Forward secrecy	HKDF ratchet (one-way key derivation)
Post-compromise security	PQC re-keying via fresh ML-KEM encapsulation
Rollback resistance	Counter comparison with rejection of older values
Domain separation	Separate send/receive ratchet chains
Message authentication	AES-GCM with counter-based AAD

Forward secrecy analysis: Each ratchet step applies HKDF-SHA-256 to derive the next key. Since HKDF is a one-way function, knowing K_n does not allow computation of K_{n-1} . An adversary who compromises a ratchet key cannot decrypt past messages.

Re-keying provides post-compromise security: If an adversary compromises the current ratchet state, they can only decrypt future messages until the next PQC re-key. The re-key uses a fresh ML-KEM encapsulation, which is quantum-safe, establishing a completely new key tree.

10.3 Attack Analysis

10.3.1 Brute Force on PQENC Packet

Attack: Try all possible 256-bit AES keys.

Difficulty: 2^{256} operations. Even with a quantum computer using Grover’s algorithm, the cost is 2^{128} operations, which is infeasible.

Verdict: → Infeasible

10.3.2 KEM Private Key Recovery

Attack: Recover the ML-KEM-1024 private key from the public key or KEM ciphertext.

Difficulty: Requires solving the Module-LWE problem with parameters providing $> 2^{248}$ classical security. No known classical or quantum algorithm can solve this efficiently.

Verdict: → Infeasible

10.3.3 Signature Forgery

Attack: Forge a valid signature on a chosen message.

Difficulty:

- ML-DSA-87: Requires $> 2^{248}$ classical operations.

- SLH-DSA-256: Requires $> 2^{248}$ classical operations (hash collision resistance).
- FN-DSA-1024: Requires $> 2^{248}$ classical operations.

Verdict: $\rightarrow$ Infeasible

10.3.4 MITM on Signaling Server

Attack: The signaling server substitutes FIPS keys during the PQ handshake.

Difficulty: The commit-reveal protocol forces the attacker to commit to keys before seeing the other party's keys. The PoP signature proves ownership. The SAS verification requires the attacker to produce matching SAS codes, which has probability 2^{-64} per attempt. The SAS must be verified out-of-band (e.g., phone call), which the attacker cannot control.

Verdict: $\rightarrow$ Infeasible (assuming SAS is verified)

10.3.5 Replay Attack

Attack: Replay a captured PQENC packet.

Difficulty: The timestamp validation rejects packets with timestamps deviating more than ± 180 seconds from the current time. The 16-byte random nonce ensures uniqueness.

Limitation: Within the 180-second window, replay is theoretically possible if the recipient does not maintain a replay cache. This is a known limitation mitigated by the short window.

Verdict: $\rightarrow$ Mitigated (short window), but not eliminated for same-session replay

10.3.6 Ratchet Compromise and Backward Recovery

Attack: Compromise a ratchet key and attempt to recover past messages.

Difficulty: HKDF is a one-way function. Given $K_n = \text{HKDF}(K_{n-1}, \text{counter}_n)$, computing K_{n-1} from K_n is as hard as inverting SHA-256, which is computationally infeasible (2^{256} operations).

Verdict: $\rightarrow$ Infeasible

10.3.7 Ratchet Rollback

Attack: Send a message with a lower ratchet counter to revert the receiver's state.

Difficulty: The receiver explicitly checks for rollback: if $\text{received_counter} < \text{current_counter}$ (and the difference is not a wraparound), the message is rejected. This is enforced with a signed comparison that accounts for uint32 wraparound at 2^{32} .

Verdict: $\rightarrow$ Prevented

10.3.8 AES-GCM Nonce Reuse

Attack: Cause nonce reuse under the same key.

Difficulty: Each encryption uses a cryptographically random 12-byte IV (96 bits). Under a single ratchet key, the probability of nonce collision is approximately $n^2/2^{96}$ where n is the number of encryptions. For $n = 2^{32}$ (the maximum counter value), this gives approximately 2^{-32} , which is negligible.

Verdict: $\rightarrow$ Infeasible within the ratchet lifecycle

10.3.9 GCM Tag Forgery

Attack: Modify ciphertext or AAD and produce a valid authentication tag.

Difficulty: AES-GCM provides 128-bit authentication tag security. The probability of successful forgery per attempt is 2^{-128} .

Verdict: $\rightarrow$ Infeasible

10.3.10 Side-Channel Attacks in Browser

Attack: Timing side-channel on commitment comparison.

Mitigation: The commitment comparison uses constant-time XOR reduction:

```
let mismatch = hashBytes.length !== commitmentBytes.length ? 1 : 0;
for (let i = 0; i < hashBytes.length; i++)
  mismatch |= hashBytes[i] ^ commitmentBytes[i];
```

Limitation: JavaScript JIT compilation may introduce timing variations. True constant-time execution in a browser is not guaranteed. However, the practical exploitability of this over a network connection is extremely low (microsecond-level timing differences, obscured by network jitter).

Verdict: → Mitigated (constant-time code), theoretical risk remains due to JIT

10.3.11 Memory Extraction

Attack: Extract cryptographic keys from browser memory.

Difficulty: Modern browsers sandbox JavaScript execution. Memory access requires either a browser exploit or operating system compromise, both of which are outside the threat model of the cryptographic protocol itself.

Mitigation: Keys are stored as `Uint8Array` and `CryptoKey` objects. The `CryptoKey` objects created by the Web Crypto API are non-extractable (created with `extractable: false` implicitly in some paths), meaning they cannot be exported via JavaScript even by the same origin.

Verdict: → Application-level keys can be read from JS variables if the browser is compromised; `CryptoKey` objects are protected by the browser's key storage.

10.3.12 Supply Chain Attack on Libraries

Attack: Modify the @noble/post-quantum library files to introduce backdoors.

Mitigation: All script and modulepreload elements include Subresource Integrity (SRI) SHA-512 hashes. The browser will refuse to execute any script whose hash does not match the expected value.

Verdict: → Mitigated by SRI (assuming the initial HTML is delivered correctly)

10.3.13 Clickjacking / UI Redress

Attack: Embed the application in an iframe and trick the user into performing actions.

Mitigation: The Content Security Policy includes `frame-ancestors 'none'`, and the X-Frame-Options: DENY header prevents framing.

Verdict: → Prevented

10.3.14 DoS via Chat RAM Exhaustion

Attack: Send large volumes of chat messages to exhaust the recipient's memory.

Mitigation: A 2 GB RAM limit is enforced. When the limit is reached, incoming messages are blocked and the peer is notified. The chat can be cleared to free memory. Individual messages are limited to 65,536 characters.

Verdict: → Mitigated (rate-limited, not eliminated)

Attack Vector	Difficulty	Status
AES-256 brute force	2^{256} classical/ 2^{128} quantum	Infeasible
ML-KEM key recovery	2^{248+} classical	Infeasible
Signature forgery	2^{248+} classical	Infeasible
MITM (SAS verified)	2^{64} per attempt	Infeasible
Replay (outside window)	N/A	Prevented
Replay (within window)	Requires same session	Mitigated
Ratchet backward recovery	2^{256} (inverting SHA-256)	Infeasible
Ratchet rollback	Explicit check	Prevented
GCM nonce collision	2^{-32} at counter limit	Negligible
GCM tag forgery	2^{-128} per attempt	Infeasible
Timing side-channel	JIT-dependent	Mitigated
Memory extraction	Requires browser/OS compromise	Mitigated
Supply chain (SRI)	Requires HTML compromise	Mitigated
Clickjacking	CSP + X-Frame-Options	Prevented
RAM exhaustion DoS	2 GB limit + notifications	Mitigated

10.4 Security Summary Matrix

11 Implementation Details

11.1 Technology Stack

Component	Technology
PQC Algorithms	@noble/post-quantum (local builds)
Symmetric Crypto	Web Crypto API (browser-native)
P2P Communication	PeerJS (WebRTC wrapper)
Media Encryption	WebRTC Insertable Streams + Web Worker
File Streaming	File System Access API (Chrome/Edge)
UI Framework	Vanilla HTML/CSS/JS + Tailwind CSS
Font	Space Grotesk (UI), JetBrains Mono (code)

11.2 Memory Management

The application implements several memory management strategies:

- **Chat RAM tracking:** Every message adds estimated byte usage. When the 2 GB limit is reached, incoming messages are blocked.
- **Chat history pruning:** Messages beyond 1,024 DOM elements are removed, with corresponding RAM accounting.
- **Blob URL lifecycle:** All created `URL.createObjectURL()` URLs are tracked and revoked when the chat is cleared.
- **Stream-to-disk:** Large files (>2 GB) are only supported with the File System Access API, writing chunks directly to disk.
- **File chunk buffer management:** When saving to disk, chunks are queued and written sequentially to avoid memory spikes.

11.3 Data Processing Chain

Incoming DataChannel messages are processed sequentially using a promise chain:

```
_dataProcChain = _dataProcChain.then(() => handleIncomingData(data));
```

This prevents race conditions when processing multiple messages that modify shared state (e.g., ratchet counters, file transfer state).

11.4 Keyboard Shortcuts

Shortcut	Action
Ctrl+1	Switch to Key Generation tab
Ctrl+2	Switch to Encrypt + Sign tab
Ctrl+3	Switch to Decrypt + Verify tab
Ctrl+4	Switch to WebRTC P2P tab
Ctrl+5	Switch to PQ Handshake tab
Ctrl+6	Switch to Video Call tab
Enter	Send chat message

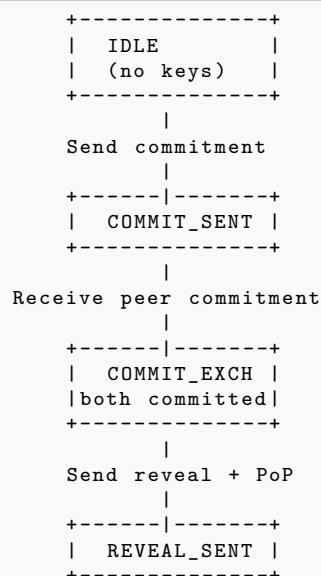
11.5 Accessibility

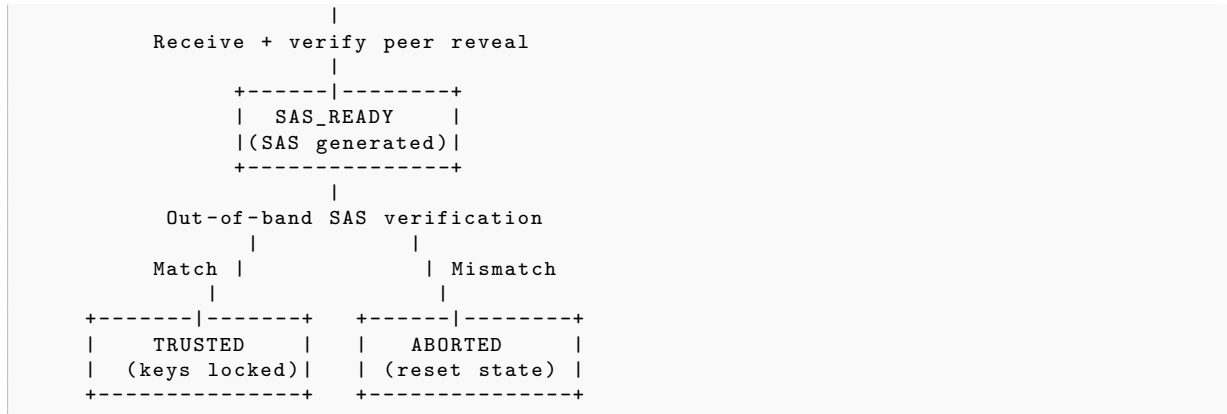
The application includes:

- ARIA roles (`role="tab"`, `role="tabpanel"`, `aria-selected`)
- Focus-visible outlines (`:focus-visible` with accent color)
- `prefers-reduced-motion` support (disables animations)
- Semantic HTML structure

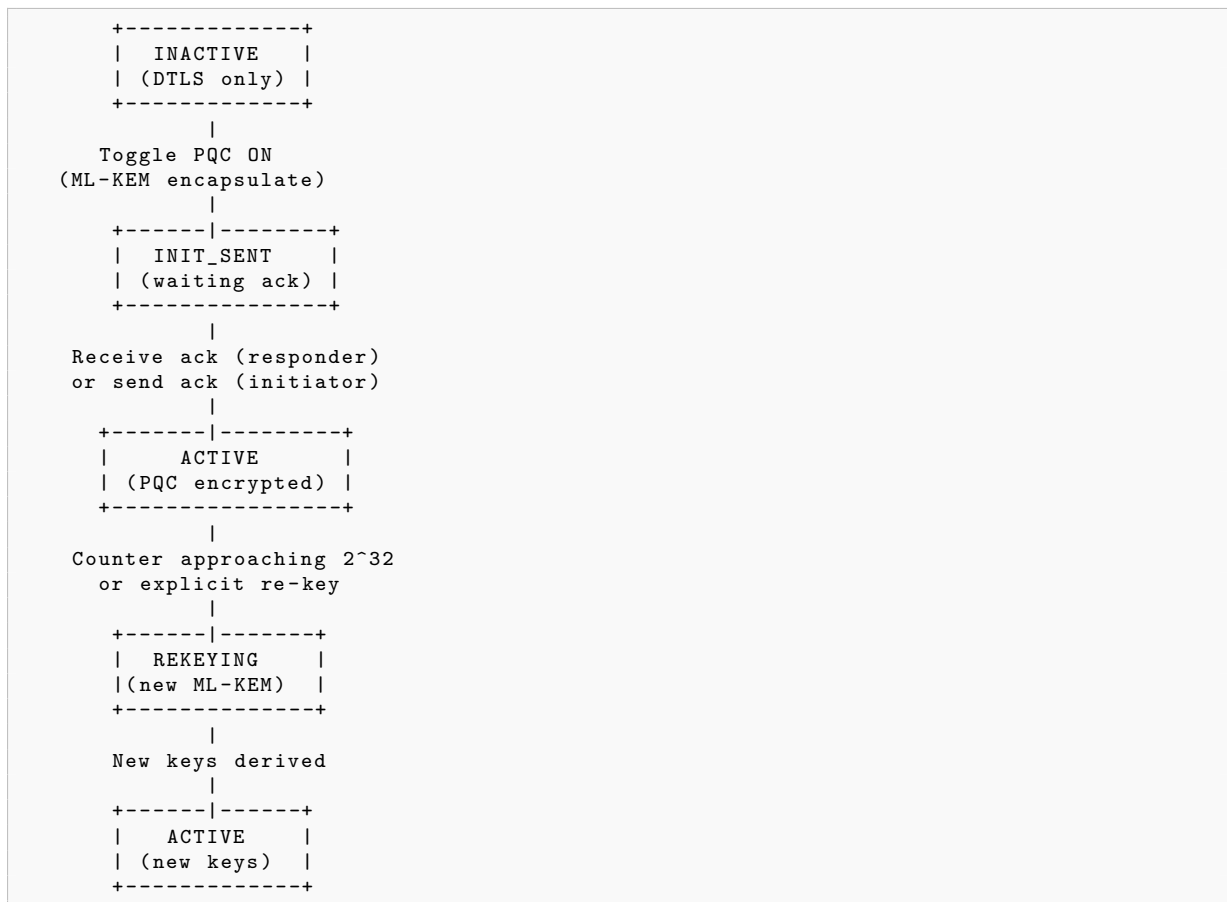
12 Protocol State Machines

12.1 PQ Handshake State Machine

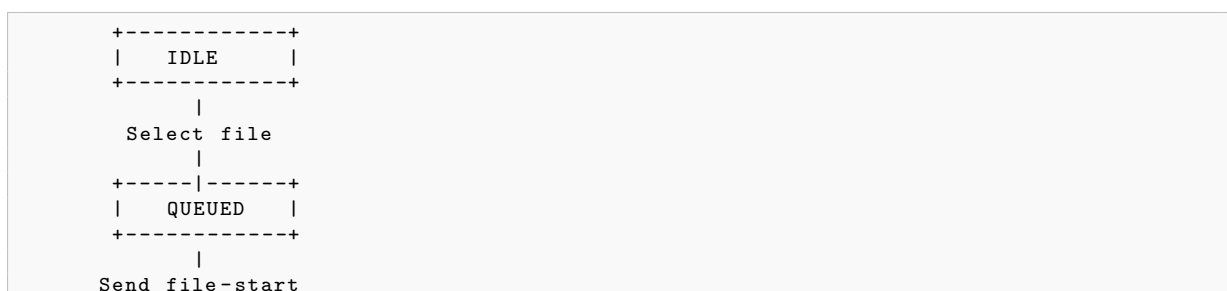




12.2 PQC Session State Machine



12.3 File Transfer State Machine





13 Comparison with Related Work

Feature	0x8	Signal	PGP/GPG
KEM Algorithm	ML-KEM-1024 (PQC)	X3DH (classical)	RSA/ElGamal (classical)
Signature Algorithm	ML-DSA/SLH-DSA/FN-DSA (PQC)	Ed25519	RSA/DSA
Forward Secrecy	HKDF ratchet + PQC re-key	Double ratchet	None (by default)
MITM Protection	Commit-reveal + SAS (64-bit)	Safety numbers	Key fingerprint
PQC Media Encryption	Insertable Streams+AES-GCM ratchet	SRTP (classical)	N/A
NIST Level	5 (highest)	N/A (classical)	Variable
Browser-native	Yes	No (app)	No (native)
Standard Compliance	FIPS 203/204/205/206	Custom protocol	RFC 4880

14 Limitations and Future Work

14.1 Known Limitations

- **WebRTC DTLS fallback:** Without PQC session encryption enabled, the underlying DTLS layer uses classical ECDH, which is vulnerable to SNDL.
- **Replay within timestamp window:** PQENC packets within the ± 180 -second window could theoretically be replayed. A per-session nonce cache would eliminate this.
- **TURN credential obscurity:** The obfuscation of TURN credentials is not cryptographic protection.
- **Browser memory limits:** JavaScript `Uint8Array` indices are limited to 32-bit integers, constraining individual array sizes to ~ 4 GB.
- **No group encryption:** The current protocol supports only 1:1 communication.
- **Signaling server dependency:** PeerJS requires a signaling server for initial connection establishment. The PQ handshake mitigates MITM risks but does not eliminate the dependency.

14.2 Future Directions

- **Hybrid KEM:** Combine ML-KEM-1024 with X25519 to provide defense-in-depth against potential weaknesses in lattice assumptions.
- **Double ratchet:** Implement a full Signal-style double ratchet with DH ratcheting for stronger post-compromise security.
- **Group messaging:** Extend to support PQC group encryption using techniques from the MLS protocol.
- **Decentralized signaling:** Replace PeerJS with a decentralized signaling mechanism (e.g., libp2p, WebTorrent DHT).

15 Conclusion

0x8 demonstrates that post-quantum cryptographic communication at NIST Level 5 security is feasible in a browser-native application without installation, plugins, or server-side key management. The combination of ML-KEM-1024 key encapsulation, multiple NIST-standardized digital signature algorithms, AES-256-GCM authenticated encryption with signature-in-AAD binding, a commit-reveal handshake with SAS verification, forward-secret ratcheted sessions, and post-quantum media encryption via Insertable Streams provides a comprehensive defense against both classical and quantum adversaries.

The primary security guarantee—quantum-safe confidentiality and authenticity—rests on the computational hardness of lattice problems (Module-LWE/SIS for ML-KEM and ML-DSA), hash function security (for SLH-DSA), and NTRU lattice problems (for FN-DSA), all at the highest NIST security level. An attacker attempting to compromise the system would face computational costs exceeding 2^{192} operations even with a quantum computer, making attacks infeasible for the foreseeable future.

The most critical security dependency is the out-of-band SAS verification: users who skip this step are vulnerable to MITM attacks by a malicious signaling server. Future work should explore more ergonomic verification mechanisms and formal verification of the protocol designs.

References

- [1] National Institute of Standards and Technology. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard. 2024.
- [2] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard. 2024.
- [3] National Institute of Standards and Technology. FIPS 205: Stateless Hash-Based Digital Signature Standard. 2024.
- [4] National Institute of Standards and Technology. FIPS 206: Fast-Fourier Lattice-Based Compact Signature over NTRU Standard. 2024.
- [5] Krawczyk, H. Cryptographic Extraction and Key Derivation: The HKDF Scheme. CRYPTO 2010.
- [6] Dworkin, M. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM). NIST SP 800-38D, 2007.

- [7] Perrig, A., Song, D. Hash Visualization: A New Technique to Improve Real-World Security. International Workshop on Cryptographic Techniques and E-Commerce, 1999. (SAS concept)
- [8] Vaudenay, S. Secure Communications over Insecure Channels Based on Short Authenticated Strings. CRYPTO 2005.
- [9] Marlinspike, M. The Double Ratchet Algorithm. Open Whisper Systems, 2016.
- [10] Paul Miller, @noble/cryptography. noble-post-quantum: ML-KEM, ML-DSA, SLH-DSA, FN-DSA for JavaScript.